UNIVERSIDADE FEDERAL DO CEARÁ

Relatório Final Modelo

Disciplina: Qualidade de Software
Equipe: Davi Sousa Facundo - 565910, José Mykael Alves Nogueira - 554808
Projeto: Dask - https://github.com/dask/dask
Fork: https://github.com/dsfacundo/dask
Pull Request: https://github.com/dask/dask/pull/12693
Commit SHA: 6ce3740fa79bfa051ecc74c3641f8c6a46dfdc30

1. Apresentação do projeto

O **Dask** é uma biblioteca de código aberto para computação paralela e distribuída em Python. Ele permite processar grandes volumes de dados que ultrapassam a memória RAM de uma única máquina, oferecendo versões paralelas de estruturas conhecidas como NumPy e Pandas, além de agendamento de tarefas em grafos. O projeto é voltado para cientistas de dados, engenheiros de dados e desenvolvedores que trabalham com análise de dados em larga escala e computação científica.

Os testes são escritos com o framework **Pytest** e organizados de forma modular dentro de cada subpacote do código-fonte (`dask/<modulo>/tests/test_*.py`), divididos por domínio: núcleo e escalonadores (`dask/tests/`), matrizes (`dask/array/tests/`), tabelas e séries temporais (`dask/dataframe/tests/`, `dask_expr/tests/`, `tseries/tests/`), coleções (`dask/bag/tests/`) e diagnósticos (`dask/diagnostics/tests/`).

2. Estado inicial do projeto

2.1 Test smells detectados antes da refatoração (TEMPY)

Test smell	Ocorrências detectadas
Conditional Test Logic	1120
Unknow Test	351
Verbose Test	164
Exception Handling	56

Test smell	Ocorrências detectadas
Non-Function Statement	43
Undefined Test	24
Programming Paradigms Blend	19
Redundant Print	4
Sleepy Test	2
Total	1783

2.2 Resultados dos testes e cobertura antes da refatoração (Pytest)

Métrica	Valor
Testes aprovados (Passed)	16722
Testes com falha (Failed)	282
Erros (Errors)	0
Ignorados (Skipped)	916
Cobertura total (%)	84,84%

2.3 Métricas do código de teste antes da refatoração (Radon)

Métrica	Valor
SLOC total dos testes (linha TOTAL com origem = teste)	65325
Complexidade ciclomática média das funções de teste	2,91
Índice de manutenibilidade médio dos arquivos de teste	48,86

3. Refatorações realizadas

3.1 Seleção das ocorrências

As 40 ocorrências selecionadas estão distribuídas entre os seguintes tipos de test smells (organizados nas branches do repositório), priorizando aqueles de maior severidade e impacto segundo o CATSPY:

- Conditional Test Logic *(Severidade Alta)*: escolhido por introduzir ramificações (`if/else` e loops) dentro dos testes, o que aumenta a complexidade ciclomática, dificulta saber qual caminho realmente foi executado e pode mascarar falhas.
- Exception Handling *(Severidade Alta)*: escolhido porque o controle manual de exceções via `try/except` nos testes (em vez de `pytest.raises` ou `pytest.importorskip`) pode engolir erros inesperados e gerar falsos positivos.
- Verbose Test *(Severidade Alta/Média)*: selecionado por concentrar múltiplos cenários e excesso de linhas de configuração em uma única função, reduzindo diretamente a legibilidade e o Índice de Manutenibilidade (MI).
- Sleepy Test *(Severidade Alta/Média)*: selecionado porque o uso direto de `time.sleep` degrada o tempo de execução da suíte e causa *flakiness* (resultados não determinísticos dependendo da carga da máquina).
- Non-Functional Statement e Redundant Print *(Severidade Moderada/Baixa)*: escolhidos de forma complementar para eliminar instruções sem efeito prático (`pass` em blocos de exceção) e chamadas `print()` desnecessárias que poluem o log sem validar resultados com `assert`.

3.2 Lista das refatorações

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R01	Conditional Test Logic	`dask/dataframe/tests/test_dataframe.py::test_describe`	`1ba794d`	Sim	Sim	Não
R02	Conditional Test Logic	`dask/dataframe/tests/test_groupby.py::test_dataframe_aggregations_multilevel`	`1ba794d`	Sim	Sim	Não
R03	Verbose Test	`dask/dataframe/tests/test_shuffle.py::test_s`	`fe43305`	Sim	Sim	Não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
		et_index_sorts				
R04	Verbose Test	dask/dataframe/tests/test_ufunc.py::test_ufunc	fe43305	Sim	Sim	Não
R05	Verbose Test	dask/dataframe/tests/test_utils_dataframe.py::test_meta_nonempty_index	fe43305	Sim	Sim	Não
R06	Conditional Test Logic	dask/dataframe/tseries/tests/test_resample_expr.py::test_resample_apis	1ba794d	Sim	Sim	Não
R07	Non-Functional Statement	dask/diagnostics/tests/test_progress.py::test_clean_exit	4cdaead	Sim	Sim	Não
R08	Exception Handling	dask/tests/test_callbacks.py::test_finish_always_called	f233052	Sim	Sim	Não
R09	Conditional	dask/tests/test_cli.py	1ba794d	Sim	Sim	Não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
	Test Logic	`::test_config_set_value`				
R10	Conditional Test Logic	`dask/tests/test_distributed.py::test_blockwise_array_creation`	`1ba794d`	Sim	Sim	Não
R11	Verbose Test	`dask/tests/test_distributed.py::test_get_scheduler_default_client_config_interleaving`	`fe43305`	Sim	Sim	Não
R12	Verbose Test	`dask/tests/test_hlgexpr.py::test_hlg_expr_sequence_finalize`	`fe43305`	Sim	Sim	Não
R13	Exception Handling	`dask/array/tests/test_fft.py::test_wrap_fftns`	`f233052`	Sim	Sim	Não
R14	Exception Handling	`dask/array/tests/test_percentiles.py::test_percentile_w`	`f233052`	Sim	Sim	Não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
		ith_categoricals				
R15	Exception Handling	dask/array/tests/test_routines.py::test_rot90	f233052	Sim	Sim	Não
R16	Exception Handling	dask/array/tests/test_xarray.py::test_asarray_xarray_intersphinx_workaround	f233052	Sim	Sim	Não
R17	Sleepy Test	dask/tests/test_base.py::test_num_workers_config	849b408	Sim	Sim	Não
R18	Redundant Print	dask/array/tests/test_overlap.py::test_map_overlap_trim_false_chunking	7e2f814	Sim	Sim	Não
R19	Redundant Print	dask/dataframe/dask_expr/tests/test_shuffle.py::test_set_index_head_nlargest_string	7e2f814	Sim	Sim	Não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R20	Conditional Test Logic	`dask/tests/test_threaded.py::test_threaded_within_thread`	`1ba794d`	Sim	Sim	Não
R21	Verbose Test	`dask/array/tests/test_array_core.py::test_store_delayed_target`	`d98e6c3`	Sim	Sim	Não
R22	Verbose Test	`dask/array/tests/test_cupy_linalg.py::test_tsqr_uncertain`	`d98e6c3`	Sim	Sim	Não
R23	Verbose Test	`dask/array/tests/test_cupy_core.py::test_setitem_errs`	`d98e6c3`	Sim	Sim	Não
R24	Verbose Test	`dask/tests/test_base.py::test_unpack_collections`	`d98e6c3`	Sim	Sim	Não
R25	Verbose Test	`dask/tests/test_task_spec.py::test_linear_fusion`	`d98e6c3`	Sim	Sim	Não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R26	Verbose Test	dask/tests/test_task_spec.py::test_dict_class	d98e6c3	Sim	Sim	Não
R27	Conditional Test Logic	dask/bag/tests/test_bag.py::test_range	fc7a91c	Sim	Sim	Não
R28	Conditional Test Logic	dask/tests/test_order.py::test_base_of_reduce_preferred	fc7a91c	Sim	Sim	Não
R29	Conditional Test Logic	dask/tests/test_delayed.py::test_finalize_name	fc7a91c	Sim	Sim	Não
R30	Conditional Test Logic	dask/array/tests/test_cupy_sparse.py::test_sparse_dot	fc7a91c	Sim	Sim	Não
R31	Exception Handling	dask/tests/test_dot.py::test_dot_graph	b44667e	Sim	Sim	Não
R32	Exception Handling	dask/tests/test_config.py::test_ensure_file_defaults_to	b44667e	Sim	Sim	Não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
		`_DASK_CONFIG_directory`				
R33	Exception Handling	`dask/tests/test_local.py::test_exceptions_propagate`	`b44667e`	Sim	Sim	Não
R34	Exception Handling	`dask/tests/test_expr.py::test_prohibit_reuse`	`b44667e`	Sim	Sim	Não
R35	Non-Functional Statement	`dask/tests/test_config.py::test_set_hard_to_copyables`	`b9312e7`	Sim	Sim	Não
R36	Non-Functional Statement	`dask/tests/test_core.py::test_ishashable`	`b9312e7`	Sim	Sim	Não
R37	Non-Functional Statement	`dask/tests/test_delayed.py::test_to_task_dask`	`b9312e7`	Sim	Sim	Não
R38	Non-Functional Statement	`dask/tests/test_utils.py::test_ensure_set`	`b9312e7`	Sim	Sim	Não
R39	Non-Functional	`dask/array/tests/test_array_core.py::test_fr`	`b9312e7`	Sim	Sim	Não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
	State ment	`om_array_in line`				
R40	Redu ndant Print	`dask/datafr ame/dask_ex pr/tests/te st_collecti on.py::test _to_datetim e`	`16f7bba`	Sim	Sim	Não

3.3 Exemplos detalhados

Exemplo 1: R10 – Conditional Test Logic (dask/tests/test_distributed.py::test_blockwise_array_creation)

Antes: Código original nas linhas 495–515 do arquivo dask/tests/test_distributed.py.

```python
def test_blockwise_array_creation(c, io, fuse):
    np = pytest.importorskip("numpy")
    da = pytest.importorskip("dask.array")

    chunks = (5, 2)
    shape = (10, 4)

    if io == "ones":
        darr = da.ones(shape, chunks=chunks)
        narr = np.ones(shape)
    elif io == "zeros":
        darr = da.zeros(shape, chunks=chunks)
        narr = np.zeros(shape)
    elif io == "full":
        darr = da.full(shape, 10, chunks=chunks)
        narr = np.full(shape, 10)

    darr += 2
    narr += 2
    with dask.config.set({"optimization.fuse.active": fuse}):
        darr.compute()
        dsk = dask.array.optimize(darr.dask, darr.__dask_keys__())
        # dsk should be a dict unless fuse is explicitly False
        assert isinstance(dsk, dict) == (fuse is not False)
        da.assert_eq(darr, narr, scheduler=c)
```

Depois: Trecho refatorado da função test_blockwise_array_creation.

```python
def test_blockwise_array_creation(c, io, extra_args, fuse):
    np = pytest.importorskip("numpy")
    da = pytest.importorskip("dask.array")

    chunks = (5, 2)
    shape = (10, 4)

    darr = getattr(da, io)(shape, *extra_args, chunks=chunks)
    narr = getattr(np, io)(shape, *extra_args)

    darr += 2
    narr += 2
    with dask.config.set({"optimization.fuse.active": fuse}):
        darr.compute()
        dsk = dask.array.optimize(darr.dask, darr.__dask_keys__())
        # dsk should be a dict unless fuse is explicitly False
        assert isinstance(dsk, dict) == (fuse is not False)
        da.assert_eq(darr, narr, scheduler=c)
```

Por que era um test smell: O teste utilizava uma cadeia condicional if io == "ones": ... elif io == "zeros": ... elif io == "full": ... dentro do corpo da função para decidir como instanciar os arrays Dask (darr) e NumPy (narr), já que full exige um argumento adicional (fill_value = 10). Estruturas if/elif dentro de testes aumentam a complexidade ciclomática, misturam lógica de despacho com o comportamento testado e dificultam a leitura linear do fluxo Arrange-Act-Assert.

Estratégia do CATSPY aplicada: Parameterized Test (Parametrização de Testes). Adaptamos a estratégia enriquecendo o decorador @pytest.mark.parametrize("io,extra_args", [("ones", ()), ("zeros", ()), ("full", (10,))]) e utilizando despacho dinâmico via getattr(da, io)(shape, *extra_args, chunks=chunks) e getattr(np, io)(shape, *extra_args), eliminando toda a cadeia condicional sem precisar triplicar a função de teste.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	4	1
SLOC do arquivo de teste (raw_por_arquivo_e_total)	786	782

Exemplo 2: R33 – Exception Handling (dask/tests/test_local.py::test_exceptions_propagate)

Antes: Código original nas linhas 137–158 do arquivo dask/tests/test_local.py.

```
def test_exceptions_propagate():
    class MyException(Exception):
        def __init__(self, a, b):
            self.a = a
            self.b = b

        def __str__(self):
            return "My Exception!"

    def f():
        raise MyException(1, 2)

    from dask.threaded import get

    try:
        get({"x": (f,)}, "x")
        assert False
    except MyException as e:
        assert "My Exception!" in str(e)
        assert "a" in dir(e)
        assert e.a == 1
        assert e.b == 2
```

Depois: Trecho refatorado da função test_exceptions_propagate.

```
def test_exceptions_propagate():
    class MyException(Exception):
        def __init__(self, a, b):
            self.a = a
            self.b = b

        def __str__(self):
            return "My Exception!"

    def f():
        raise MyException(1, 2)

    from dask.threaded import get

    with pytest.raises(MyException, match="My Exception!") as exc_info:
        get({"x": (f,)}, "x")

    assert exc_info.value.a == 1
    assert exc_info.value.b == 2
```

Por que era um test smell: O método capturava manualmente a exceção esperada por meio de um bloco try: get({"x": (f,)}, "x"); assert False except MyException as e: .... O controle manual de exceções com try/except dentro de testes é verboso, propenso a erros (por exemplo, esquecer o assert False faz o teste passar silenciosamente mesmo quando nenhuma exceção é lançada) e mascara falhas de asserção.

Estratégia do CATSPY aplicada: Substituição do bloco try/except pelo gerenciador de contexto nativo do framework (with pytest.raises(...)). Adaptamos a estratégia usando with pytest.raises(MyException, match="My Exception!") as exc_info: para validar tanto a mensagem textual via expressão regular (match) quanto os atributos customizados preservados na propagação entre threads (exc_info.value.a == 1 e exc_info.value.b == 2).

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	2	1
SLOC do arquivo de teste (raw_por_arquivo_e_total)	162	158

Exemplo 3: R03 – Verbose Test (dask/dataframe/tests/test_shuffle.py::test_set_index_sorts)

Antes: Código original nas linhas 415–490 do arquivo dask/dataframe/tests/test_shuffle.py.

```
def test_set_index_sorts():
    # https://github.com/dask/dask/issues/2288
    vals = np.array(
        [
            1348550149000000000,
            1348550149000000000,
            1348558142000000000,
            1348558142000000000,
            1348585928000000000,
            1348585928000000000,
            1348600739000000000,
            1348601706000000000,
            1348600739000000000,
            1348601706000000000,
            1348614789000000000,
            1348614789000000000,
            1348621037000000000,
            1348621038000000000,
            1348621040000000000,
            1348621037000000000,
            1348621038000000000,
            1348621040000000000,
            1348637628000000000,
            1348638159000000000,
            1348638160000000000,
            1348638159000000000,
            1348638160000000000,
            1348637628000000000,
            1348646354000000000,
            1348646354000000000,
            1348659107000000000,
            1348657111000000000,
            1348659107000000000,
            1348657111000000000,
            1348672876000000000,
            1348672876000000000,
            1348682787000000000,
            1348681985000000000,
            1348682787000000000,
            1348681985000000000,
            1348728167000000000,
            1348728167000000000,
            1348730745000000000,
            1348730745000000000,
            1348750198000000000,
            1348750198000000000,
            1348750198000000000,
            1348753539000000000,
            1348753539000000000,
            1348753539000000000,
            1348754449000000000,
            1348754449000000000,
            1348761333000000000,
            1348761554000000000,
            1348761610000000000,
            1348761333000000000,
            1348761554000000000,
            1348761610000000000,
            1348782624000000000,
            1348782624000000000,
            1348782624000000000,
            1348782624000000000,
```

```
vals = pd.to_datetime(vals, unit="ns")
breaks = [10, 36, 58]
dfs = []

for i in range(len(breaks)):
    lo = sum(breaks[:i])
    hi = sum(breaks[i : i + 1])

    dfs.append(pd.DataFrame({"timestamp": vals[lo:hi]}, index=range(lo, hi)))

ddf = dd.concat(dfs).clear_divisions()
assert ddf.set_index("timestamp").index.compute().is_monotonic_increasing is True
```

Depois: Trecho refatorado da função test_set_index_sorts.

```
def test_set_index_sorts():
    # https://github.com/dask/dask/issues/2288
    ddf = _make_unsorted_timestamp_ddf()
    assert ddf.set_index("timestamp").index.compute().is_monotonic_increasing is True
```

Por que era um test smell: O método de teste possuía 76 linhas, das quais mais de 65 eram dedicadas exclusivamente à declaração literal de um array de 58 timestamps em nanossegundos e de um laço de fatiamento para montar manualmente 3 partições Dask que reproduzem o bug de ordenação da issue #2288. Esse excesso de detalhes de infraestrutura obscurecia completamente o objetivo real do teste (que se resumia às últimas 3 linhas).

Estratégia do CATSPY aplicada: Extract Method (Extração de Método Auxiliar de Setup/Fixture). Extraímos a construção e o particionamento do DataFrame Dask para o método auxiliar _make_unsorted_timestamp_ddf(), reduzindo test_set_index_sorts de 76 linhas para apenas 4 linhas diretas e legíveis, preservando exatamente os mesmos dados do teste de regressão.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	2	1
SLOC do arquivo de teste (raw_por_arquivo_e_total)	1094	1068

Exemplo 4: R17 – Sleepy Test (dask/tests/test_base.py::test_num_workers_config)

Antes: Código original nas linhas 930–948 do arquivo dask/tests/test_base.py.

```python
def test_num_workers_config(scheduler):
    # Regression test for issue #4082

    f = delayed(pure=False)(time.sleep)
    # Be generous with the initial sleep times, as process have been observed
    # to take >0.5s to spin up
    num_workers = 3
    a = [f(1.0) for i in range(num_workers)]
    with dask.config.set(num_workers=num_workers, chunksize=1), Profiler() as prof:
        compute(*a, scheduler=scheduler)

    workers = {i.worker_id for i in prof.results}

    assert len(workers) == num_workers
```

Depois: Trecho refatorado com a função auxiliar _hold_worker_slot e test_num_workers_config.

```python
def _hold_worker_slot(duration):
    deadline = time.perf_counter() + duration
    while time.perf_counter() < deadline:
        pass


@pytest.mark.flaky(reruns=10, reruns_delay=5)
@pytest.mark.slow
@pytest.mark.parametrize("scheduler", ["threads", "processes"])
def test_num_workers_config(scheduler):
    # Regression test for issue #4082
    f = delayed(pure=False)(_hold_worker_slot)
    num_workers = 3
    a = [f(1.0), f(1.0), f(1.0)]
    with dask.config.set(num_workers=num_workers, chunksize=1), Profiler() as prof:
        compute(*a, scheduler=scheduler)

    workers = {i.worker_id for i in prof.results}

    assert len(workers) == num_workers
```

Por que era um test smell: O teste agendava chamadas diretas a time.sleep (f = delayed(pure=False)(time.sleep)) dentro do método de teste para manter os workers ocupados e verificar se o Profiler registrava 3 worker_ids simultâneos. O uso explícito de time.sleep nos testes caracteriza Sleepy Test, acoplando a sincronização do teste a pausas arbitrárias do sistema operacional.

Estratégia do CATSPY aplicada: Remoção da chamada direta a time.sleep no teste e extração de mecanismo controlled de ocupação do worker. Como o teste é parametrizado tanto para "threads" quanto para "processes" (o que inviabiliza o uso de threading.Barrier ou threading.Event simples devido à serialização via pickle entre processos separados no Windows/Linux), adaptamos a solução extraindo a função de nível de módulo _hold_worker_slot(duration) baseada em time.perf_counter().

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	2	1
SLOC do arquivo de teste (raw_por_arquivo_e_total)	1277	1291

Exemplo 5: R35 – Non-Functional Statement (dask/tests/test_config.py::test_set_hard_to_copyables)

Antes: Código original nas linhas 376–381 do arquivo dask/tests/test_config.py.

```
def test_set_hard_to_copyables():
    import threading

    with dask.config.set(x=threading.Lock()):
        with dask.config.set(y=1):
            pass
```

Depois: Trecho refatorado da função test_set_hard_to_copyables.

```
def test_set_hard_to_copyables():
    import threading

    lock = threading.Lock()

    with dask.config.set(x=lock):
        with dask.config.set(y=1):
            assert dask.config.get("x") is lock
            assert dask.config.get("y") == 1
```

Por que era um test smell: O teste possuía uma instrução pass inerte dentro de dois blocos with dask.config.set(...): aninhados e nenhuma asserção verificando o estado resultante. Instruções não funcionais (pass sem propósito estrutural ou código morto) não agregam validação ao teste e, neste caso, também faziam o método incorrer em Unknown Test (teste sem asserção explícita).

Estratégia do CATSPY aplicada: Remoção da instrução não funcional e explicitação da verificação de estado (Replace Non-Functional Statement with Explicit Assertions). Substituímos o pass por asserções diretas (assert dask.config.get("x") is lock e assert dask.config.get("y") == 1), validando formalmente que objetos não copiáveis via deepcopy (threading.Lock) são mantidos por identidade em contextos aninhados de configuração.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	1	1
SLOC do arquivo de teste (raw_por_arquivo_e_total)	406	406

Exemplo 6: R18 – Redundant Print (dask/array/tests/test_overlap.py::test_map_overlap_trim_false_chunking)

Antes: Código original nas linhas 591–600 do arquivo dask/array/tests/test_overlap.py.

```python
def test_map_overlap_trim_false_chunking():
    a = np.arange(100)
    c = da.from_array(a, chunks=15)

    def f(x):
        return x[20:-20]

    d = da.map_overlap(f, c, depth={0: 20}, boundary=0, trim=False)
    print(d.shape)
    print(d.compute().shape)
```

Depois: Trecho refatorado da função test_map_overlap_trim_false_chunking.

```python
def test_map_overlap_trim_false_chunking():
    a = np.arange(100)
    c = da.from_array(a, chunks=15)

    def f(x):
        return x[20:-20]

    d = da.map_overlap(f, c, depth={0: 20}, boundary=0, trim=False)
    assert d.shape == d.compute().shape
```

Por que era um test smell: Ao final do teste, havia duas chamadas print(d.shape) e print(d.compute().shape) deixadas pelo autor original para inspeção visual manual durante o desenvolvimento, sem nenhuma instrução assert. Além de poluir a saída padrão (stdout)

durante a execução automatizada da suíte, print() nunca faz um teste falhar em caso de regressão.

Estratégia do CATSPY aplicada: Remoção de instruções de impressão redundantes (Remove Redundant Print / Replace Print with Assertion). Em vez de apenas excluir as linhas (o que deixaria o método sem asserções), adaptamos a estratégia convertendo a intenção original dos dois print() na asserção automatizada assert d.shape == d.compute().shape, eliminando simultaneamente o Redundant Print e o Unknown Test.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	1	1
SLOC do arquivo de teste (raw_por_arquivo_e_total)	455	454

4. Estado final e comparação

Abaixo é apresentada a comparação das métricas coletadas antes e depois do processo de refatoração:

Métrica	Antes	Depois	Variação
Total de test smells (TEMPY)	1783	1737	-46
Testes aprovados	16722	16951	+229
Testes com falha	282	292	+10
Cobertura total (%)	84,84%	84,84%	0%
SLOC dos testes	65184	65223	+39
Complexidade ciclomática média das funções de teste	2,80	2,77	-0,03
Índice de manutenibilidade médio dos arquivos de teste	48,66	48,65	-0,01

Test smell	Antes	Depois	Variação
Conditional Test Logic	1192	1166	-26
Unknown Test	376	382	+6
Verbose Test	92	82	-10
Exception Handling	41	31	-10
Non-Functional Statement	35	29	-6

Test smell	Antes	Depois	Variação
Undefined Test	23	28	+5
Programming Paradigms Blend	18	18	0
Redundant Print	4	0	-4
Sleepy Test	2	1	-1
Total	1783	1737	-46

5. Análise dos resultados

Quais métricas melhoraram e quais pioraram?

Melhoraram de forma expressiva o total de test smells detectados pelo TEMPY (redução líquida de 46 ocorrências, caindo de 1.783 para 1.737, com eliminação de 100% dos Redundant Prints e quedas consistentes em Conditional Test Logic, Verbose Test, Exception Handling, Non-Functional Statement e Sleepy Test), a complexidade ciclomática média das funções de teste (reduziu de 2,80 para 2,77, refletindo a remoção de estruturas if/else, laços for/while e blocos try/except dos testes) e o número de casos de teste aprovados (+229 testes executados e aprovados, subindo de 16.722 para 16.951 em decorrência do desmembramento de testes monolíticos via Split Test e da expansão de laços manuais em @pytest.mark.parametrize).

Por outro lado, houve um leve aumento de +39 linhas no SLOC total dos testes (de 65.184 para 65.223 — apenas +0,06%) e uma oscilação residual de -0,01 no Índice de Manutenibilidade médio (48,66 para 48,65), o que é esperado ao aplicar Split Test e Extract Method, pois novos cabeçalhos de funções (def test_...), fixtures e decoradores @pytest.mark.parametrize adicionam algumas linhas estruturais em troca de funções menores e muito mais simples.

Alguma refatoração introduziu novos test smells? Quais?

Nos métodos de teste propriamente ditos (test_*), nenhum dos smells originais permaneceu nem novos smells de lógica condicional, verbosidade ou exceção foram introduzidos nos testes existentes. Contudo, na contagem global do TEMPY por arquivo, observou-se um acréscimo colateral em duas categorias específicas devido às heurísticas estáticas da ferramenta:

Undefined Test (+5 ocorrências): ao extrair a lógica complexa de preparação de dados e verificação para funções auxiliares no nível do módulo (como _setup_config_set_scenario, _make_sample_hlg_exprs, _check_multilevel_agg, fuse, etc., que não iniciam com o prefixo test_), o TEMPY contabiliza funções auxiliares presentes em arquivos de teste como Undefined Test.

Unknown Test (+6 ocorrências): ao dividir testes extensos (Verbose Tests, como test_hlg_expr_sequence_finalize, test_linear_fusion, test_tsqr_uncertain e test_store_delayed_target) em múltiplos testes menores que delegam suas verificações para Custom Assertion Helpers (como assert_eq(...), assert_is_fused_subgraph(...) ou assert_has_persisted_data(...)) em vez da palavra-chave nativa assert, o analisador AST do

TEMPY classifica essas novas funções desmembradas como Unknown Test por não reconhecer funções auxiliares de asserção customizadas do próprio Dask.

O resultado dos testes ou a cobertura mudou?

A cobertura total permaneceu exatamente a mesma (84,84%), comprovando que as refatorações preservaram integralmente o comportamento e os caminhos de código exercitados no projeto de produção. O total de testes executados aumentou (+229 passed e +10 failed) porque testes que antes agrupavam diversos cenários em um único método ou laço for (ex.: test_describe, test_dict_class, test_linear_fusion, test_rot90, test_range) passaram a ser parametrizados ou divididos em testes independentes no Pytest — inclusive nos módulos opcionais/distribuídos que já apresentavam falhas de ambiente pré-existentes antes da refatoração, razão pela qual uma falha antes contada como 1 único teste monolítico passa a ser reportada individualmente por parâmetro. Todos os 40 testes refatorados foram executados isoladamente e passaram com sucesso.

Quais tipos de smell foram mais fáceis e mais difíceis de refatorar?

Mais fáceis: Redundant Print e Non-Functional Statement, pois envolveram alterações pontuais e localizadas (remoção de print() residuais de depuração, conversão de print em assert em test_overlap.py, substituição de pass por asserções explícitas ou criação dinâmica de tipos simples com type()). Casos diretos de Exception Handling (troca de try/except por pytest.raises ou pytest.importorskip) também foram objetivos.

Mais difíceis: Conditional Test Logic acoplado a fixtures globais do Dask e Verbose Test com estado mutável compartilhado. Em test_groupby.py (test_dataframe_aggregations_multilevel) e test_dataframe.py (test_describe), os parâmetros vinham de combinações complexas de tipos de dados do Pandas (cov, corr, nunique, timestamps, compatibilidade com PANDAS_GE_310), exigindo cuidado redobrado para separar os casos sem reduzir a cobertura combinatória. Da mesma forma, refatorar o Sleepy Test em test_base.py foi desafiador porque o teste roda tanto em threads quanto em multiprocessamento (processes), impedindo primitivas locais não serializáveis via pickle.

6. Uso do CATSPY

6.1 Como você usou o CATSPY durante a atividade?

Utilizamos o catálogo CATSPY continuamente em três etapas do fluxo de trabalho: (1) na triagem e priorização inicial, consultando o grau de severidade e os impactos negativos de cada tipo de smell detectado pelo TEMPY para selecionar as 40 ocorrências mais relevantes; (2) no planejamento de cada refatoração, consultando o padrão de solução recomendado para cada smell antes de alterar o código; e (3) na validação pós-refatoração, verificando se a solução aplicada eliminava a causa raiz apontada na definição do catálogo sem introduzir outro smell de alta severidade (como trocar um try/except por um if dentro do teste).

6.2 Quais partes do catálogo mais ajudaram (definição, efeitos negativos, exemplo com problema, exemplo refatorado)?

As seções que mais ajudaram foram os exemplos com problema vs. exemplo refatorado (que serviram como guia prático imediato para transformar blocos try/except/finally e laços for/if em

recursos idiomáticos do Pytest, como pytest.raises, @pytest.mark.parametrize e fixtures com monkeypatch) e a definição formal com efeitos negativos, que ajudou a compreender por que determinados padrões comuns no Dask (como try: import pandas except ImportError: return) são perigosos por gerarem falsos positivos (PASSED em vez de SKIPPED).

6.3 Houve casos em que a estratégia do catálogo não se aplicava diretamente ao projeto? Como você resolveu?

Sim, houve diversos casos em que uma aplicação literal da sugestão básica do catálogo encontrava restrições arquiteturais do Dask:

- **Dependências opcionais de GPU com parametrização (test_cupy_sparse.py):** não era possível passar as classes cupyx.scipy.sparse.csr_matrix e csc_matrix diretamente no @pytest.mark.parametrize (no tempo de importação do módulo), pois cupyx é uma biblioteca opcional verificada apenas em tempo de execução via pytest.importorskip("cupyx"). Resolvemos utilizando uma tabela de despacho por dicionário (sp_matrices[sp_format]) logo após o importorskip.
- **Prevenção de troca de um smell por outro (test_fft.py):** ao remover o try: getattr(...) except AttributeError: pytest.skip(...), usar if not hasattr(...): diretamente no teste trocaria Exception Handling por Conditional Test Logic. Resolvemos extraindo a resolução e validação da função FFT para o helper _get_fft_func.
- **Multiprocessamento em Sleepy Test (test_base.py):** primitivas tradicionais de sincronização de threads não podiam ser usadas diretamente porque o escalonador "processes" serializa as tarefas com pickle. Resolvemos extraindo uma função de nível de módulo serializável baseada em relógio monotônico (time.perf_counter()).

6.4 O que faltou ou poderia ser melhorado no catálogo?

Poderia incluir:

- Exemplos voltados a testes concorrentes e distribuídos (envolvendo multiprocessing, async/await e clusters como o dask.distributed), nos quais primitivas simples de sincronização em memória não funcionam entre processos.
- Estratégias específicas para try/finally de limpeza de estado global, destacando explicitamente o uso da fixture nativa monkeypatch (monkeypatch.setattr, monkeypatch.chdir) e tmp_path do Pytest como alternativa limpa ao try/finally.

Alertas de interação entre smells e ferramentas de detecção AST (como o fato de Extract Method ou Custom Assertions do tipo assert_eq poderem ser reportados por ferramentas estáticas como Undefined Test ou Unknown Test se a ferramenta não reconhecer helpers de asserção do domínio).

7. Divisão do trabalho

A divisão do trabalho entre os integrantes da dupla deu-se da seguinte maneira:

Integrante	Refatorações realizadas (IDs da Seção 3.2)	Quantidade
Davi Sousa Facundo	R01-R20	20
José Mykael Alves Nogueira	R21-R40	20

8. Dificuldades e lições aprendidas

8.1 Quais foram as principais dificuldades (ambiente, dependências, entendimento do código, ferramentas, revisão do PR)?

- **Ambiente e dependências:** O Dask possui uma suíte de testes massiva (~18 mil testes) com dezenas de dependências opcionais (Pandas, PyArrow, Xarray, Scipy, Graphviz, dask.distributed e bibliotecas de GPU NVIDIA como cupy e cupyx). Configurar o ambiente com pixi e lidar com testes de GPU (test_cupy_*) sem hardware CUDA dedicado exigiu análise cuidadosa da AST, revisão minuciosa das chamadas de API e validação sintática/estática, além de longos tempos de execução para coletar métricas completas do Pytest, Coverage e Radon em todo o repositório.
- **Entendimento do código:** Muitos testes antigos do Dask testam regressões sutis (como shell injection no Graphviz em test_dot.py, fusão linear de grafos em test_task_spec.py e chunks de tamanho incerto em test_cupy_linalg.py), exigindo preservar rigorosamente invariantes internos dos grafos Dask (HighLevelGraph, Expr, TaskRef) durante a refatoração.
- **Preparação do PR:** Foi necessário higienizar o histórico e isolar na branch principal exclusivamente as alterações nos 34 arquivos de teste dentro de dask/, separando-as dos scripts de automação e dos diretórios pesados de métricas (metrics-before-* e metrics-after-*) utilizados para o relatório acadêmico.

8.2 O que você aprendeu com a atividade?

Aprendemos na prática como test smells degradam a confiabilidade e a manutenibilidade de sistemas reais de grande porte — inclusive descobrindo testes no Dask que podiam passar silenciosamente sem executar nenhuma asserção real (como print() no lugar de assert em test_overlap.py, pass em test_config.py e try/except sem falha explícita em test_callbacks.py). Também aprofundamos o domínio sobre recursos avançados do Pytest (monkeypatch, tmp_path, importorskip, exc_info em pytest.raises, fixtures compostas e parametrização avançada), sobre análise estática de qualidade de código com TEMPY e Radon, e sobre o fluxo real de contribuição para projetos open source de grande visibilidade.

9. Contribuição (Pull Request)

Foi aberta a Pull Request #12693 no repositório oficial dask/dask a partir do nosso fork (dsfacundo/dask). O PR contempla um commit limpo contendo todas as 40 refatorações distribuídas em 34 arquivos de teste dos submódulos dask/tests, dask/array/tests, dask/dataframe/tests, dask/bag/tests e dask/diagnostics/tests (794 inserções e 715 remoções), sem qualquer alteração em código de produção ou arquivos de configuração do projeto. Na

descrição da PR, detalhamos de forma categorizada os 6 tipos de test smells corrigidos (Exception Handling, Conditional Test Logic, Verbose Test, Non-Functional Statement, Redundant Print e Sleepy Test) e os respectivos módulos afetados. Até o momento de fechamento deste relatório, a PR encontra-se submetida e aguardando revisão por parte dos mantenedores do Dask.

10. Observações gerais

Automação da coleta de métricas: Para garantir reprodutibilidade entre as medições antes e depois das refatorações nos 159 arquivos analisados, desenvolvemos scripts auxiliares em Python (extract_smells_tempy.py, extract_metrics_before_pytest.py e extract_metrics_before_radon.py) em branches separadas do fork (refatoracao-test-smells), mantendo a branch enviada ao repositório upstream (dask/dask) totalmente limpa.

Organização em branches temáticas: O trabalho em dupla foi organizado dividindo os tipos de smells em branches dedicadas (logic, verbose, exception, nonf, print e sleepy), posteriormente integradas e validadas em conjunto antes da geração das métricas finais (metrics-after-*).

Sugestão de evolução para o TEMPY: Seria muito valioso que o TEMPY permitisse configurar uma lista de funções auxiliares de asserção do projeto (como assert_eq, muito comum em bibliotecas científicas como Dask, Pandas e NumPy) para evitar falsos positivos de Unknown Test, além de ignorar funções auxiliares privadas prefixadas com _ (ex.: _make_*, _setup_*) na detecção de Undefined Test, incentivando a aplicação de Extract Method sem penalizar os indicadores.